

Deep Reinforcement Learning for Electric Vehicle Routing Optimization

Energy-Minimizing Electric Vehicle Routing Problem (EM-EVRP)

Dhia Elhak Ezzeddini Salem Fradi

Higher School of Communication of Tunis – Carthage University

Supervisor: Dr. Ferdaoues Chaabane **Host:** Sagemcom

Academic Year 2025–2026

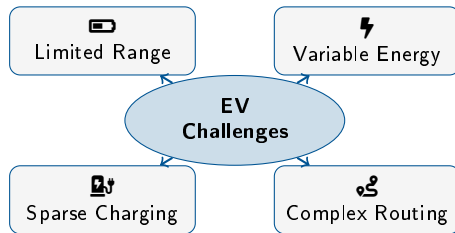
Context: EV Adoption & Logistics Challenges

Global Trends

- 14M+ EVs sold in 2023
- 40M+ expected by 2030
- Corporate sustainability push

Core Challenge

Limited range & sparse charging $\Rightarrow$ **smart routing required**



Our Contribution

End-to-end DRL solution + physics-based energy model + real-world deployment

Energy-Minimizing EVRP (EM-EVRP)

Objective

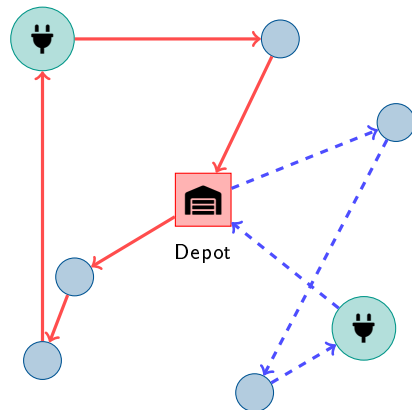
Minimize **total energy consumption** while serving all customers

Key Constraints:

- Each customer visited once
- Vehicle capacity limits
- Battery SOC ≥ 0 always
- Optional charging stops

NP-Hard

Exact methods don't scale!



Multi-vehicle routes with charging stops

Why Minimize Energy Instead of Distance?

Key Insight: Shortest path $\neq$ most energy-efficient path!

A longer downhill route may use *less* energy than a shorter uphill one.

Energy depends on:

- **Vehicle load** (heavier = more energy)
- **Road gradient** (uphill vs downhill)
- **Speed profile** & aerodynamic drag
- **Regenerative braking**

Benefits of Energy Optimization

- Extended range per charge
- Fewer charging stops
- Lower operational costs
- Reduced battery degradation

Physics-Based Energy Model

Mechanical Power Required on Arc (i,j)

$$P_{ij} = \underbrace{\frac{1}{2} C_d \rho A v_{ij}^2}_{\text{Aerodynamic drag}} + \underbrace{m_{ij} g \sin \alpha_{ij}}_{\text{Gravitational}} + \underbrace{m_{ij} g C_r \cos \alpha_{ij}}_{\text{Rolling resistance}}$$

Energy with Efficiency

$$E_{ij} = \begin{cases} \phi_d \varphi_d P_{ij} t_{ij} & P_{ij} \geq 0 \\ \phi_r \varphi_r P_{ij} t_{ij} & P_{ij} < 0 \end{cases}$$

Negative power = regeneration!

Key Parameters

m_{ij}	Vehicle + cargo mass
α_{ij}	Road slope angle
C_d, C_r	Drag coefficients
ϕ, φ	Efficiencies

$$\text{Objective: } \min \sum_k \sum_{(i,j)} E_{ij} \cdot x_{ijk}$$

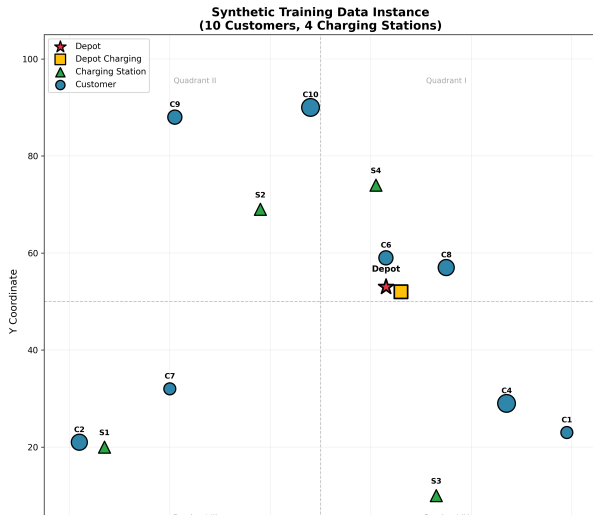
Training Data Generation

Synthetic Data

- Random coordinates on 100×100 grid
- Random elevations for slopes
- Random customer demands
- Pairwise distances & energy costs

Dataset Size

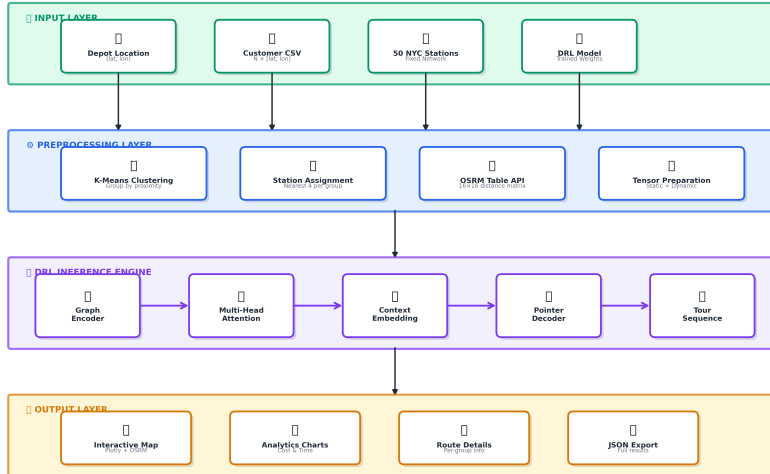
Training	1,280,000
Validation	10,000
Batch size	1,024



End-to-End System Pipeline

EV Route Optimization System Architecture

Deep Reinforcement Learning with Real Street Routing



Open Source Routing Machine:

- Production-grade routing engine
- Real street-level distances
- OpenStreetMap data

Pipeline:

- 1 Upload customer GPS coordinates
- 2 Query OSRM for distance matrix
- 3 Run DRL inference
- 4 Visualize on real map

Scalability: K-means Clustering

- Group customers into batches
- 10 customers per cluster
- Run DRL on each cluster
- Combine results

NYC Setup

50 pre-configured charging stations across New York City

Interactive Dashboard

The screenshot displays the 'EV Route Optimization Dashboard' with a dark theme. On the left is a 'Configuration' sidebar with a futuristic car icon at the top. It includes a 'Depot Location' section with latitude and longitude input fields (40.758000, -74.06510) and a 'Customer Configuration' section with a warning that customer counts must be multiples of 10. Below these are sliders for 'Number of Customers' and 'Geographic Spread (km)', and buttons for 'Generate Customers' and 'Run Optimization (Real Streets)'. The main area has a top navigation bar with 'Solution Map', 'Analytics', 'Station Network', and 'Detailed Results'. A 'Welcome to the EV Route Optimizer!' message is followed by 'How to use:' instructions and a list of features. The 'Available Charging Station Network' section shows a map of NYC with green dots representing charging stations.

EV Route Optimization Dashboard

Deep Reinforcement Learning with Real Street Routing (DSRM + Dijkstra)

[Solution Map](#) [Analytics](#) [Station Network](#) [Detailed Results](#)

Welcome to the EV Route Optimizer!

This dashboard helps you optimize electric vehicle routes for large-scale delivery operations.

How to use:

1. Set your depot location in the sidebar
2. Generate or upload customer locations (must be multiples of 10)
3. Click "Run Optimization" to compute optimal routes

Features:

- Groups customers into exactly 10 per group (4 means clustering)
- Uses 50 fixed charging stations across NYC
- Assigns nearest 4 stations to each group centroid
- Builds distance matrix via real roads (DSRM Dijkstra)
- Deep RL model predicts optimal visit sequence
- Routes projected onto actual streets

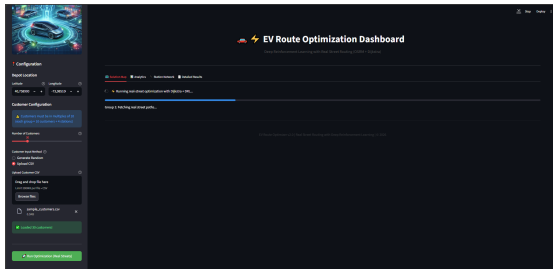
Available Charging Station Network

NYC Fixed Charging Station Network (50 Stations)

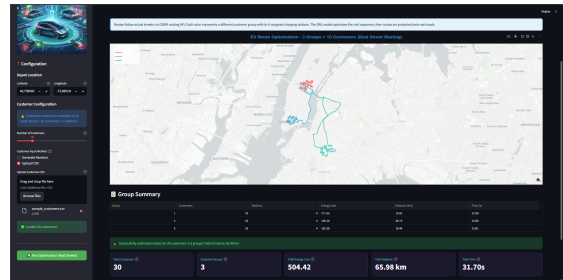
Charging Stations

Upload customers • Configure EV • Select stations • Real-time optimization

Route Optimization & Results

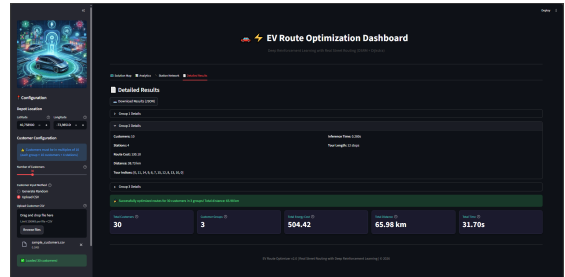
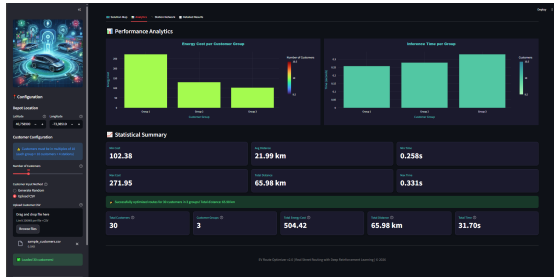


Inference Process



Optimized Routes

Analytics Dashboard



Energy breakdown • SOC tracking • Cost analysis • JSON export

Baseline Methods Overview

OR-Tools (Google)

Constraint programming + local search

Hill Climbing

Local search: 2-opt, Or-opt, Exchange
Fast but trapped in local optima

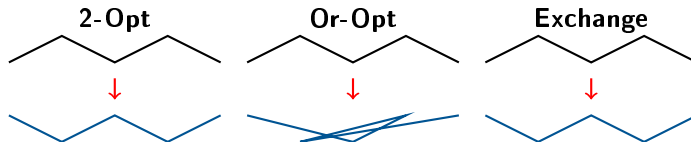
Simulated Annealing

$$P(\text{accept}) = e^{-\Delta E/T}$$

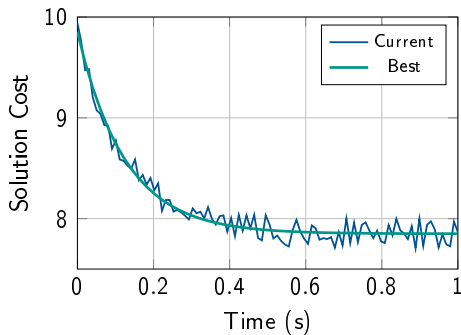
Escapes local optima via temperature

Ant Colony Optimization

Pheromone-based collective search
Good for combinatorial problems



Simulated Annealing: Convergence Behavior



Key Observations

- High variance early (exploration)
- Gradual convergence
- Track **best-ever** solution
- Plateau before time limit

Parameter Sensitivity

T_{start} and T_{end} critically affect solution quality

Why Deep Reinforcement Learning?

Traditional Limitations

- Exact: exponential time
- Heuristics: problem-specific
- No generalization
- Slow for real-time

DRL Advantages

- **Learn** to solve
- **Generalize** to new instances
- **Fast inference** (ms)
- **End-to-end** optimization

Key Idea

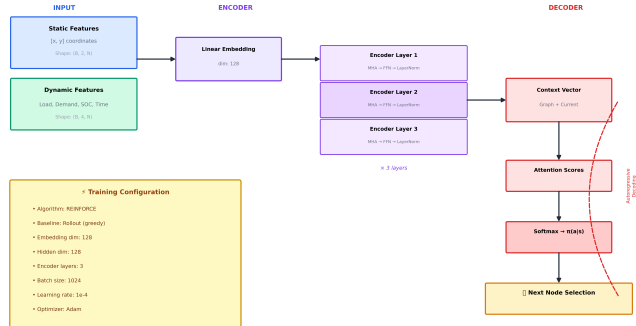
Train a neural network to **construct** solutions step-by-step, learning which decisions minimize energy consumption

Based on: Kool et al., “Attention, Learn to Solve Routing Problems!” (ICLR 2019)

Attention-Based Architecture

Attention-Based DRL Model Architecture

Encoder-Decoder with Multi-Head Attention for EVRP



Encoder

Multi-head self-attention

Decoder

Autoregressive generation

MDP Formulation & Training

MDP Components

- State** Partial solution + vehicle status
- Action** Select next node
- Reward** Negative energy consumed
- Policy** Neural network π_θ

REINFORCE with Baseline

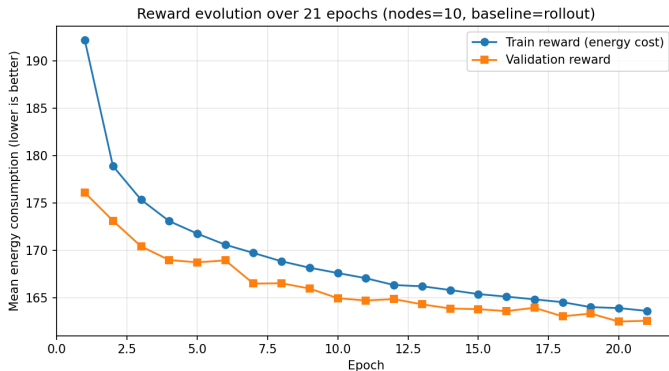
$$\nabla J \approx \frac{1}{B} \sum_i (L_i - b_i) \nabla \log p_\theta$$

Rollout baseline reduces variance

EV-Specific Features

- SOC tracking: $\text{SOC}_{t+1} = \text{SOC}_t - E_{ij}/E_{\text{battery}}$
- Load dynamics: cargo decreases after delivery
- Dynamic masking: feasibility constraints enforced

Training Results



Configuration

10 customers, 4 stations, 20 epochs, batch 1024

Observation

Consistent improvement, no overfitting

Performance Comparison

Method	Energy (kWh)	Time
OR-Tools	11.92	2.1s
lightgray DRL (Ours)	12.18	0.5s
Ant Colony	12.67	3.2s
Simulated Annealing	12.89	1.5s
Hill Climbing	13.21	0.8s

Solution Quality

- Only **2.2%** gap vs OR-Tools
- **7.8%** better than Hill Climbing

Speed

- **4x** faster than OR-Tools
- **6x** faster than ACO

Summary & Contributions

What We Built

- 1 Physics-based energy model
- 2 Baseline benchmarks (OR-Tools, SA, ACO, HC)
- 3 Attention-based DRL solver
- 4 Production-ready dashboard

Key Results

- ✓ Near-optimal solutions
- ✓ 4x faster inference
- ✓ Generalizes to new instances
- ✓ Real-world deployment ready

Future Work

Scale to 50+ customers • Multi-depot • Dynamic traffic • Fleet coordination

Thank You!

Questions?

-  W. Kool et al., *“Attention, Learn to Solve Routing Problems!”*, ICLR 2019.
-  S. Kirkpatrick et al., *“Optimization by Simulated Annealing”*, Science 1983.
-  M. Dorigo et al., *“The Ant System”*, IEEE Trans. SMC 1996.
-  A. Vaswani et al., *“Attention Is All You Need”*, NeurIPS 2017.
-  Google OR-Tools. <https://developers.google.com/optimization>
-  Project OSRM. <http://project-osrm.org/>